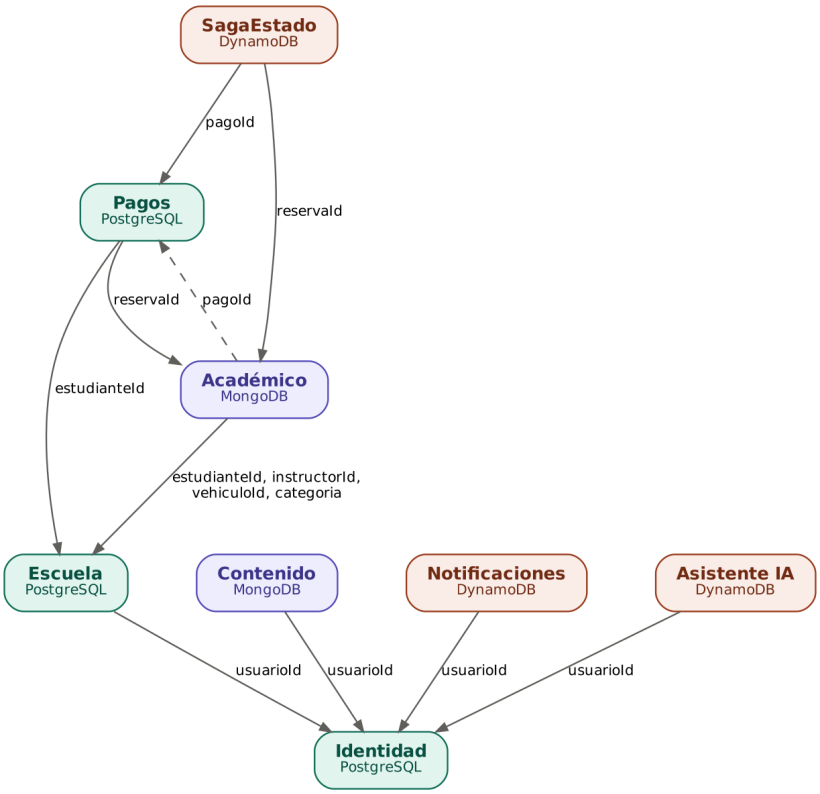
DriveHub: diseño de base de datos

Modelo de datos de los 7 microservicios más el estado de la saga, con PostgreSQL, MongoDB y DynamoDB según lo definido para la arquitectura. Cada entidad indica los casos de uso (CU) que cubre.

Vista general



Cada flecha apunta al servicio que es dueño del identificador. No hay claves foráneas entre servicios: se guarda el ID y se valida por REST al escribir o por eventos. La flecha punteada de Académico a Pagos es la referencia del pago dentro de la reserva.

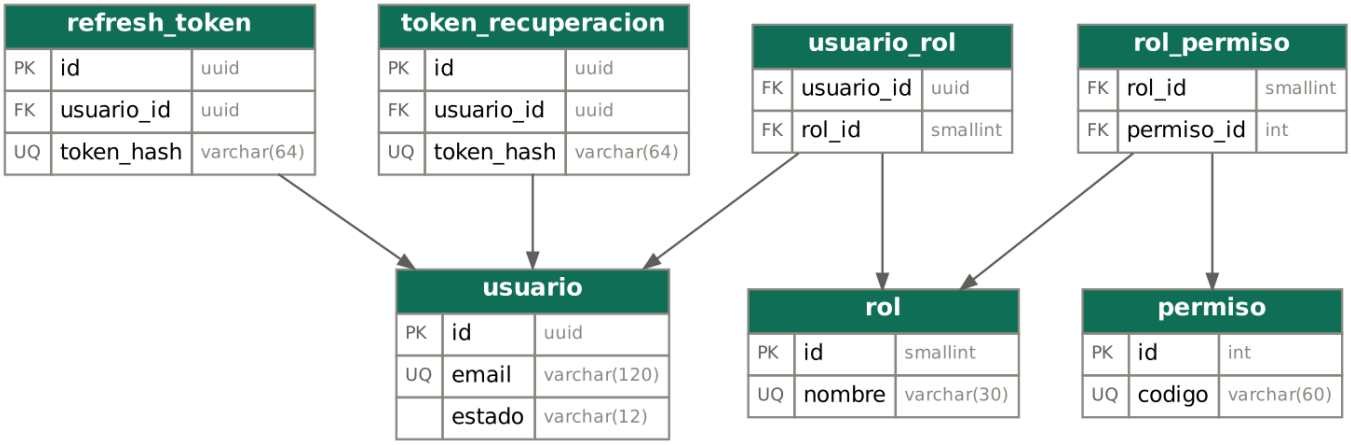
Convenciones

- **Una base por servicio:** ningún servicio lee las tablas de otro; se consultan por su API o por eventos.
- **Identificadores:** UUID v4 en PostgreSQL, ObjectId en MongoDB y ULID en las claves de DynamoDB.
- **Persona única:** el UUID de `Identidad.usuario.id` es el mismo `estudianteId`, `instructorId` y `usuarioId` en todos los servicios.
- **Fechas en UTC.** Los estados son texto con valores fijos (CHECK en PostgreSQL, validación de esquema en MongoDB) y el borrado es lógico, por estado.
- **Eventos con outbox:** Pagos y Académico guardan el evento junto al cambio de datos y un proceso aparte lo publica a RabbitMQ, para no perder eventos si el broker falla. Los consumidores son idempotentes.

Etiquetas de campo: **PK** clave primaria, **SK** clave de orden (DynamoDB), **FK** clave foránea dentro del servicio, **UQ** valor único, **REF** referencia a otro servicio (sin FK física).

1. Identidad PostgreSQL · Spring Boot

Cuentas, roles y sesiones. Es el dueño del identificador de persona: el UUID de `usuario.id` se reutiliza en todos los demás servicios como `usuarioId`, `estudianteId` o `instructorId`.



usuario Cuenta de acceso (CU01, CU02, CU06)		
PK `id`	uuid	UUID v4. Es el identificador de persona en toda la plataforma
UQ `email`	varchar(120)	Login
`password_hash`	varchar(255)	BCrypt o Argon2; nunca la contraseña en claro
`nombres`	varchar(80)	
`apellidos`	varchar(80)	
`telefono`	varchar(20)	Opcional
`foto_url`	text	Opcional; clave del archivo en el storage
`estado`	varchar(12)	ACTIVO | INACTIVO | BLOQUEADO
`intentos_fallidos`	int	Se reinicia en un login correcto; bloquea al superar el límite
`ultimo_login`	timestamptz	Opcional
`creado_en`	timestamptz	UTC
`actualizado_en`	timestamptz	UTC

rol Roles del sistema (CU05)		
PK `id`	smallint	
UQ `nombre`	varchar(30)	ESTUDIANTE | INSTRUCTOR | ADMIN
`descripcion`	varchar(120)	

permiso Permisos finos (CU05)		
PK `id`	int	
UQ `codigo`	varchar(60)	Ej: reserva.crear, pago.revertir
`descripcion`	varchar(120)	

usuario_rol Roles de cada usuario (CU01, CU05)		
FK `usuario_id`	uuid	PK compuesta. Referencia a usuario.id
FK `rol_id`	smallint	PK compuesta. Referencia a rol.id
`asignado_en`	timestamptz	

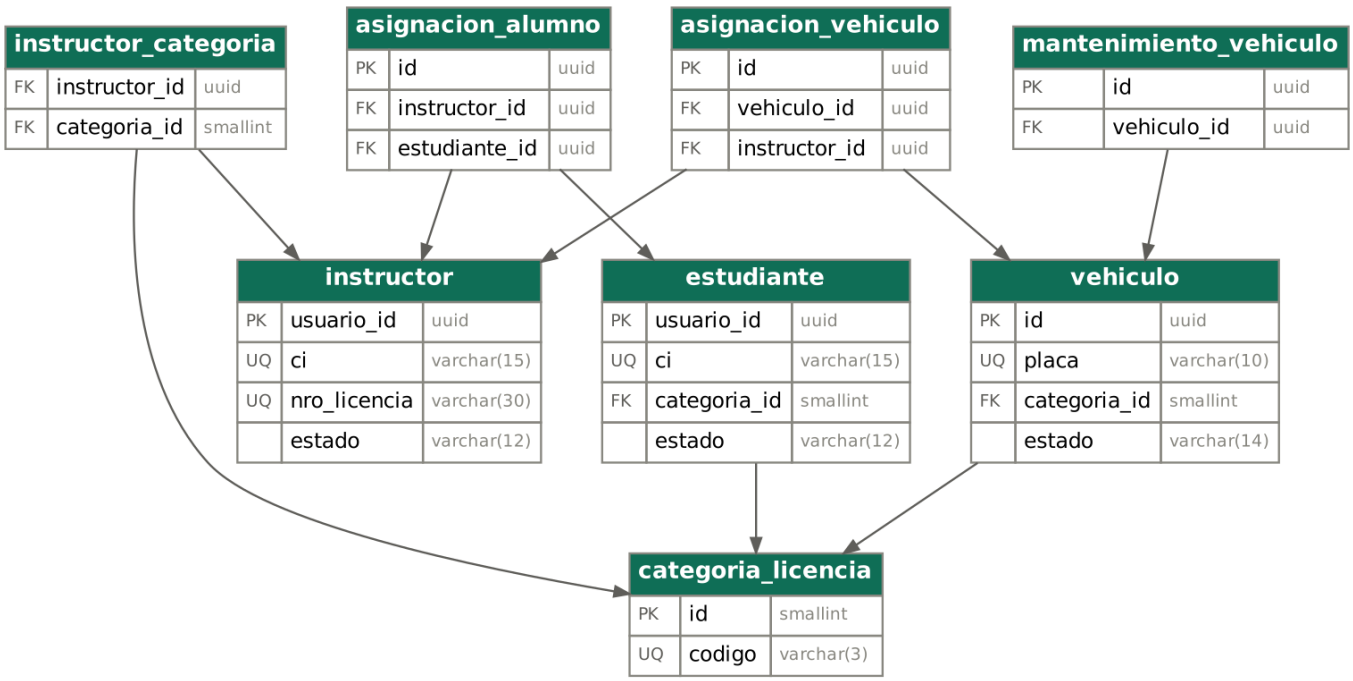
rol_permiso Permisos de cada rol (CU05)		
FK rol_id	smallint	PK compuesta. Referencia a rol.id
FK permiso_id	int	PK compuesta. Referencia a permiso.id

refresh_token Sesiones activas, login único (CU02, CU03)		
PK id	uuid	
FK usuario_id	uuid	Referencia a usuario.id
UQ token_hash	varchar(64)	SHA-256 del token; el token en claro solo lo tiene el cliente
dispositivo	varchar(10)	WEB | MOVIL
expira_en	timestamptz	
revocado	boolean	CU03 lo pone en true
creado_en	timestamptz	
Índices y reglas: Índice en (usuario_id, revocado). Un job borra los tokens vencidos.		

token_recuperacion Restablecer contraseña (CU04)		
PK id	uuid	
FK usuario_id	uuid	Referencia a usuario.id
UQ token_hash	varchar(64)	
expira_en	timestamptz	Vigencia corta (ej. 30 min)
usado	boolean	Un solo uso
creado_en	timestamptz	

2. Escuela PostgreSQL · Spring Boot

Datos de la autoescuela: estudiantes, instructores y vehículos. Los perfiles de estudiante e instructor usan como clave primaria el mismo UUID de Identidad, así no hace falta una tabla de equivalencias ni consultas adicionales.



categoria_licencia	Catálogo de categorías de licencia	
PK id	smallint	
UQ codigo	varchar(3)	A, B, C... Es el código que usan los demás servicios
descripcion	varchar(100)	
edad_minima	smallint	
clases_minimas	smallint	Mínimo de clases prácticas para rendir el examen

estudiante	Perfil del estudiante (CU08)	
PK usuario_id	uuid	= Identidad.usuario.id (sin FK física)
UQ ci	varchar(15)	Documento de identidad
fecha_nacimiento	date	
direccion	varchar(200)	
FK categoria_id	smallint	Categoría a la que aspira
estado	varchar(12)	ACTIVO | EGRESADO | RETIRADO
fecha_inscripcion	date	
observaciones	text	Opcional
creado_en	timestamptz	

instructor	Perfil del docente (CU09)	
PK usuario_id	uuid	= Identidad.usuario.id (sin FK física)
UQ ci	varchar(15)	
UQ nro_licencia	varchar(30)	
licencia_vence	date	Alerta antes del vencimiento
estado	varchar(12)	ACTIVO | INACTIVO | LICENCIA
fecha_contratacion	date	
creado_en	timestamptz	

instructor_categoria	Categorías que puede enseñar cada instructor (CU09)	
FK instructor_id	uuid	PK compuesta. Referencia a instructor.usuario_id
FK categoria_id	smallint	PK compuesta. Referencia a categoria_licencia.id

vehiculo	Autos de la escuela (CU07)	
PK id	uuid	
UQ placa	varchar(10)	
marca	varchar(40)	
modelo	varchar(40)	
anio	smallint	
color	varchar(20)	
transmision	varchar(10)	MANUAL | AUTOMATICO
FK categoria_id	smallint	Categoría en la que se usa
estado	varchar(14)	DISPONIBLE | MANTENIMIENTO | BAJA
kilometraje	int	Se actualiza al cerrar cada clase
soat_vence	date	
inspeccion_vence	date	
creado_en	timestamptz	

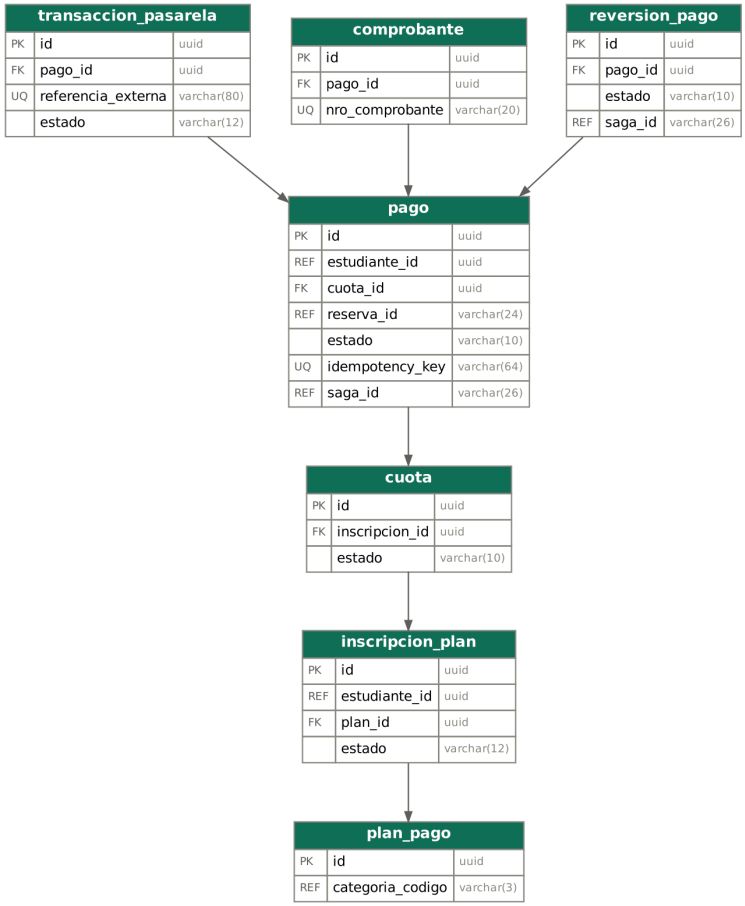
mantenimiento_vehiculo Historial de mantenimiento (CU07)		
PK `id`	uuid	
FK `vehiculo_id`	uuid	Referencia a vehiculo.id
`fecha`	date	
`tipo`	varchar(12)	PREVENTIVO | CORRECTIVO
`descripcion`	text	
`costo`	numeric(10,2)	
`km_al_momento`	int	

asignacion_vehiculo Auto asignado a un instructor (CU10)		
PK `id`	uuid	
FK `vehiculo_id`	uuid	Referencia a vehiculo.id
FK `instructor_id`	uuid	Referencia a instructor.usuario_id
`fecha_inicio`	date	
`fecha_fin`	date	NULL mientras la asignación esté vigente
Índices y reglas: Índice único parcial en (vehiculo_id) WHERE fecha_fin IS NULL: un auto tiene una sola asignación vigente.		

asignacion_alumno Instructor titular de cada estudiante (CU08, CU29)		
PK `id`	uuid	
FK `instructor_id`	uuid	Referencia a instructor.usuario_id
FK `estudiante_id`	uuid	Referencia a estudiante.usuario_id
`fecha_inicio`	date	
`fecha_fin`	date	NULL mientras esté vigente
Índices y reglas: Índice único parcial en (estudiante_id) WHERE fecha_fin IS NULL. Pagos usa esta tabla para resolver 'mis alumnos' (CU29).		

3. Pagos **PostgreSQL · Spring Boot**

Planes, cuotas y pagos con integridad ACID. Cada pago lleva una clave de idempotencia para que un reintento del cliente o de la pasarela no cobre dos veces, y los eventos salen por una tabla outbox para no perderlos si el broker falla.



plan_pago Planes que ofrece la escuela (CU25)		
PK `id`	uuid	
`nombre`	varchar(80)	
REF `categoria_codigo`	varchar(3)	Código de Escuela.categoria_licencia
`nro_clases`	smallint	Clases incluidas
`monto_total`	numeric(10,2)	
`nro_cuotas`	smallint	
`activo`	boolean	
`creado_en`	timestamptz	

inscripcion_plan Plan contratado por un estudiante (CU25, CU28)		
PK `id`	uuid	
REF `estudiante_id`	uuid	= Escuela.estudiante.usuario_id
FK `plan_id`	uuid	Referencia a plan_pago.id
`monto_total`	numeric(10,2)	Copia del monto al contratar; el plan puede cambiar después
`saldo`	numeric(10,2)	
`estado`	varchar(12)	ACTIVO | COMPLETADO | CANCELADO
`fecha_inicio`	date	

cuota Cuotas de un plan (CU25, CU28)		
PK `id`	uuid	
FK `inscripcion_id`	uuid	Referencia a inscripcion_plan.id
`nro`	smallint	
`monto`	numeric(10,2)	
`fecha_vence`	date	
`estado`	varchar(10)	PENDIENTE | PAGADA | VENCIDA | ANULADA
Índices y reglas: Único en (inscripcion_id, nro). Índice en (estado, fecha_vence) para marcar vencidas.		

pago Pago realizado (CU26, CU27, CU28, CU29)		
PK `id`	uuid	
REF `estudiante_id`	uuid	= Escuela.estudiante.usuario_id
`concepto`	varchar(10)	CUOTA | CLASE | EXAMEN
FK `cuota_id`	uuid	Opcional. Referencia a cuota.id
REF `reserva_id`	varchar(24)	Opcional. ObjectId de Académico.reservas
`monto`	numeric(10,2)	
`moneda`	char(3)	BOB por defecto
`metodo`	varchar(15)	QR | TRANSFERENCIA | EFECTIVO
`estado`	varchar(10)	PENDIENTE | CONFIRMADO | FALLIDO | REVERTIDO
UQ `idempotency_key`	varchar(64)	Evita cobros duplicados
REF `saga_id`	varchar(26)	Opcional. Clave de SagaEstado
`creado_en`	timestamptz	
`confirmado_en`	timestamptz	Opcional
Índices y reglas: Índices en (estudiante_id, creado_en DESC) y (reserva_id).		

transaccion_pasarela Intercambio con la pasarela (CU26, CU27)		
PK `id`	uuid	
FK `pago_id`	uuid	Referencia a pago.id
`proveedor`	varchar(30)	
UQ `referencia_externa`	varchar(80)	Id que devuelve la pasarela; evita procesar dos veces el webhook
`qr_payload`	text	Opcional
`estado`	varchar(12)	
`respuesta`	jsonb	Respuesta cruda de la pasarela, para auditoría
`creado_en`	timestamptz	

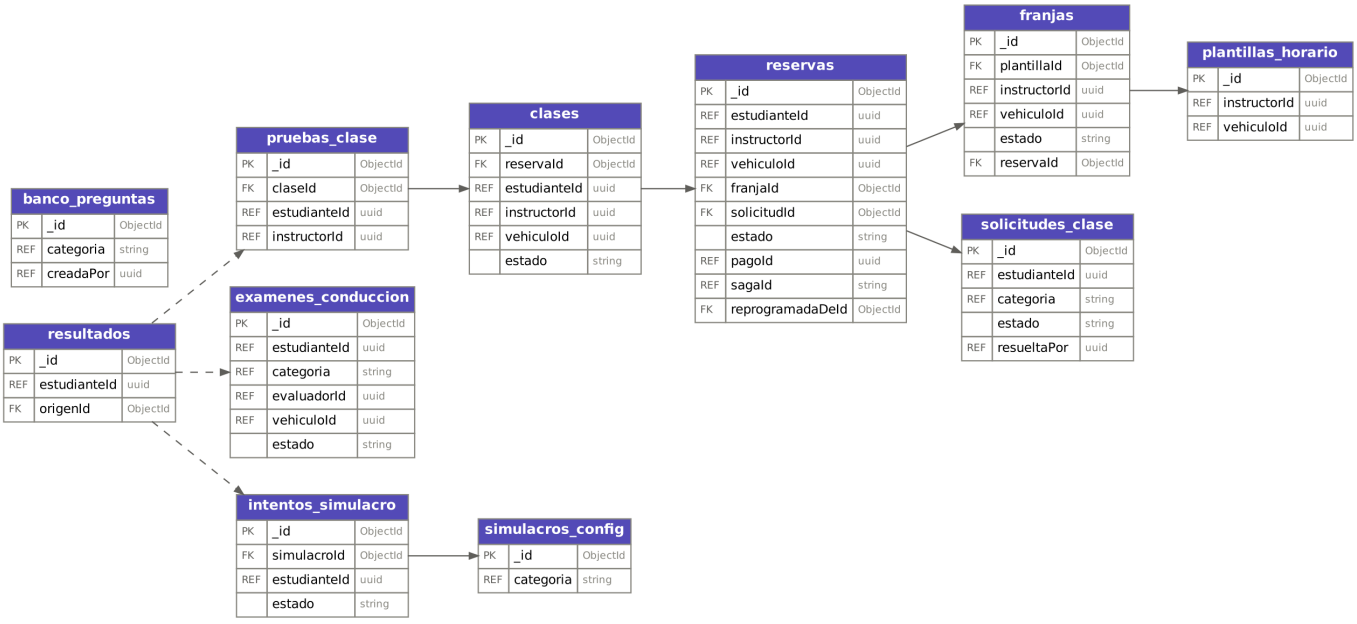
comprobante Comprobante de pago (CU30)		
PK `id`	uuid	
FK `pago_id`	uuid	Único. Referencia a pago.id
UQ `nro_comprobante`	varchar(20)	Correlativo
`monto`	numeric(10,2)	
`emitido_en`	timestamptz	
`url_pdf`	text	Clave del PDF en el storage

reversion_pago Compensación de la saga (CU31)		
PK id	uuid	
FK pago_id	uuid	Referencia a pago.id
motivo	varchar(200)	
monto	numeric(10,2)	
estado	varchar(10)	SOLICITADA | EJECUTADA | FALLIDA
REF saga_id	varchar(26)	Opcional
creado_en	timestamptz	
ejecutado_en	timestamptz	Opcional

outbox_evento Eventos pendientes de publicar (CU27, CU44)		
PK id	bigserial	
tipo	varchar(60)	pago.confirmado, pago.revertido...
payload	jsonb	
creado_en	timestamptz	
publicado_en	timestamptz	NULL hasta que RabbitMQ confirma
Índices y reglas: Se escribe en la misma transacción que el cambio de estado del pago. Un proceso aparte publica a RabbitMQ.		

4. Académico MongoDB · Spring Boot

Agenda, reservas, clases y exámenes. Las franjas horarias son documentos concretos (uno por instructor, fecha y hora) y la reserva toma una franja con una actualización atómica, así dos estudiantes no pueden quedarse con el mismo horario.



plantillas_horario	Horario semanal base del instructor (CU12)	
PK _id	ObjectId	
REF instructorId	uuid	= Escuela.instructor.usuario_id
REF vehiculoId	uuid	Escuela.vehiculo.id
diaSemana	int	1 (lunes) a 7 (domingo)
horaInicio	string	HH:mm
horaFin	string	HH:mm
duracionClaseMin	int	Ej: 60
vigenteDesde	date	
vigenteHasta	date	Opcional
activa	bool	

franjas	Horario concreto reservable (CU11, CU12, CU14, CU16)	
PK _id	ObjectId	
FK plantillaId	ObjectId	Opcional. Referencia a plantillas_horario
REF instructorId	uuid	
REF vehiculoId	uuid	
fecha	date	
horaInicio	string	HH:mm
horaFin	string	HH:mm
estado	string	LIBRE | RESERVADA | BLOQUEADA
FK reservaId	ObjectId	Opcional. Referencia a reservas
Índices y reglas: Índice único (instructorId, fecha, horaInicio) y único (vehiculoId, fecha, horaInicio). Reservar = findOneAndUpdate de estado LIBRE a RESERVADA.		

solicitudes_clase	Solicitud del estudiante (CU13)	
PK _id	ObjectId	
REF estudianteId	uuid	= Escuela.estudiante.usuario_id
REF categoria	string	Código de categoría
tipo	string	PRACTICA | TEORICA
cantidadClases	int	
preferencias	object	{ dias: [int], turno: MANANA | TARDE | NOCHE }
estado	string	PENDIENTE | APROBADA | RECHAZADA | CANCELADA
motivoRechazo	string	Opcional
REF resueltaPor	uuid	Opcional
creadaEn	date	
resueltaEn	date	Opcional

reservas Reserva de una franja (CU14, CU15, CU16)		
PK `_id`	ObjectId	
REF `estudianteId`	uuid	
REF `instructorId`	uuid	
REF `vehiculoId`	uuid	
FK `franjaId`	ObjectId	Referencia a franjas
FK `solicitudId`	ObjectId	Opcional. Referencia a solicitudes_clase
`fecha`	date	Copia de la franja para consultar sin join
`horaInicio`	string	
`horaFin`	string	
`estado`	string	PENDIENTE_PAGO | CONFIRMADA | CANCELADA | COMPLETADA | NO_ASISTIO
REF `pagoId`	uuid	Opcional. Pagos.pago.id
REF `sagaId`	string	SagaEstado.sagaId
FK `reprogramadaDeId`	ObjectId	Opcional. Reserva original si es una reprogramación
`historialEstados`	array<object>	[{ estado, en, motivo }]
`expiraPagoEn`	date	Si no se paga a tiempo, la saga la cancela y libera la franja
`creadaEn`	date	
Índices y reglas: Índices: (estudianteId, fecha), (instructorId, fecha), (estado, expiraPagoEn).		

clases Clase dictada (CU17)		
PK `_id`	ObjectId	
FK `reservaId`	ObjectId	Único. Referencia a reservas
REF `estudianteId`	uuid	
REF `instructorId`	uuid	
REF `vehiculoId`	uuid	
`fecha`	date	
`inicioReal`	date	Opcional
`finReal`	date	Opcional
`asistencia`	string	PRESENTE | TARDE | AUSENTE
`tema`	string	
`observaciones`	string	Opcional
`kmInicio`	int	
`kmFin`	int	
`estado`	string	ABIERTA | CERRADA

pruebas_clase	Evaluación práctica durante la clase (CU18)	
PK _id	ObjectId	
FK claseId	ObjectId	Referencia a clases
REF estudianteId	uuid	
REF instructorId	uuid	
tipo	string	MANEJO | ESTACIONAMIENTO | CIUDAD
criterios	array<object>	[{ codigo, descripcion, puntaje, maximo }]
puntajeTotal	double	
resultado	string	APROBADO | REPROBADO
observaciones	string	Opcional
fecha	date	

examenes_conduccion	Examen oficial de conducción (CU22)	
PK _id	ObjectId	
REF estudianteId	uuid	
REF categoria	string	
fechaProgramada	date	
REF evaluadorId	uuid	Instructor o evaluador
REF vehiculoId	uuid	
intento	int	1, 2, 3...
estado	string	PROGRAMADO | REALIZADO | CANCELADO | NO_ASISTIO
puntaje	double	Opcional
resultado	string	Opcional. APROBADO | REPROBADO
observaciones	string	Opcional

banco_preguntas	Preguntas de los simulacros (CU19)	
PK _id	ObjectId	
REF categoria	string	
tema	string	Ej: señales, prioridad de paso
tipo	string	OPCION_UNICA | OPCION_MULTIPLE | VERDADERO_FALSO
enunciado	string	
imagenUrl	string	Opcional
opciones	array<object>	[{ id, texto, correcta }]
explicacion	string	Se muestra al corregir
dificultad	int	1 a 3
activa	bool	
REF creadaPor	uuid	
creadaEn	date	
Índices y reglas: Índice en (categoria, tema, activa).		

simulacros_config	Configuración de simulacros (CU20)	
PK _id	ObjectId	
nombre	string	
REF categoria	string	
duracionMin	int	
nroPreguntas	int	
puntajeAprobacion	double	
reglas	array<object>	[{ tema, cantidad }] para armar cada intento
activo	bool	

intentos_simulacro	Intento del estudiante (CU21)	
PK _id	ObjectId	
FK simulacroId	ObjectId	Referencia a simulacros_config
REF estudianteId	uuid	
estado	string	EN_CURSO | FINALIZADO | EXPIRADO
iniciadoEn	date	
finalizadoEn	date	Opcional
preguntas	array<object>	[{ preguntaId, enunciado, opciones, elegidas, correcta }] (copia de la pregunta tal como se mostró)
aciertos	int	
puntaje	double	
aprobado	bool	
Índices y reglas: Índice en (estudianteId, iniciadoEn DESC). Las preguntas se copian dentro del intento para que editar el banco no altere intentos pasados.		

resultados	Vista consolidada de notas (CU23)	
PK _id	ObjectId	
REF estudianteId	uuid	
origen	string	SIMULACRO | PRUEBA_CLASE | EXAMEN_CONDUCCION
FK origenId	ObjectId	Documento de origen
titulo	string	
puntaje	double	
aprobado	bool	
fecha	date	
Índices y reglas: Se escribe al cerrar cada evaluación. Permite que el estudiante, el instructor y la escuela vean las notas con una sola consulta. Publica examen.calificado.		

outbox_eventos	Eventos pendientes de publicar	
PK _id	ObjectId	
tipo	string	clase.reservada, examen.calificado...
payload	object	
creadoEn	date	
publicadoEn	date	Opcional
Índices y reglas: Mismo patrón que en Pagos.		

5. Contenido MongoDB · PHP (Laravel)

Solo guarda metadatos y la ubicación del archivo; los PDF y videos viven en el storage. En Laravel se usa el paquete `mongodb/laravel-mongodb`.

manuales Manuales de la escuela (CU32, CU34)		
PK `_id`	ObjectId	
`titulo`	string	
`descripcion`	string	
REF `categoria`	string	Código de categoría o GENERAL
`tema`	string	
`version`	int	Sube al reemplazar el archivo
`archivo`	object	{ url, mime, tamanoBytes, sha256 }
`estado`	string	BORRADOR | PUBLICADO | ARCHIVADO
`etiquetas`	array<string>	
REF `publicadoPor`	uuid	= Identidad.usuario.id
`creadoEn`	date	
`actualizadoEn`	date	
Índices y reglas: Índice de texto en (titulo, descripcion, etiquetas) y en (estado, categoria).		

tutoriales Tutoriales del instructor (CU33, CU35)		
PK `_id`	ObjectId	
`titulo`	string	
`descripcion`	string	
REF `autorId`	uuid	Instructor. = Identidad.usuario.id
REF `categoria`	string	
`tipo`	string	VIDEO | PDF | ENLACE
`recurso`	object	{ url, duracionSeg }
`audiencia`	array<string>	ESTUDIANTE y/o INSTRUCTOR
`estado`	string	BORRADOR | PUBLICADO | ARCHIVADO
`etiquetas`	array<string>	
`creadoEn`	date	
Índices y reglas: Índice en (autorId, estado) y de texto en (titulo, descripcion, etiquetas).		

6. Notificaciones DynamoDB · NestJS

Historial y preferencias por usuario. Todas las consultas son por usuario y fecha, que es justo el patrón de clave de partición y de orden de DynamoDB.

Notificaciones Historial por usuario (CU36, CU38)		
PK `usuarioId`	S	Clave de partición
SK `sk`	S	'<fecha ISO>#<notifId>': ordena por fecha
`tipo`	S	RESERVA | PAGO | EXAMEN | RECORDATORIO
`canal`	S	EMAIL | PUSH | IN_APP
`titulo`	S	
`mensaje`	S	
`datos`	M	Ej: { reservaId, fecha }
`estado`	S	ENVIADA | FALLIDA
`leida`	BOOL	
`noLeidaTs`	S	Solo existe mientras no se lea: alimenta el índice GSI1
`ttl`	N	Expira a los 180 días
Índices y reglas: GSI1 'NoLeidas': PK usuarioId, SK noLeidaTs (índice disperso, lista solo las no leídas). TTL activo sobre el atributo ttl.		

PreferenciasNotificacion Preferencias del usuario (CU37)		
PK `usuarioId`	S	
`canales`	M	{ email: bool, push: bool, inApp: bool }
`tipos`	M	{ RESERVA: bool, PAGO: bool, EXAMEN: bool, RECORDATORIO: bool }
`silencioDesde`	S	Opcional. HH:mm
`silencioHasta`	S	Opcional. HH:mm
`email`	S	Copia del correo, para no consultar Identidad en cada envío
`actualizadoEn`	S	

DispositivosPush Tokens de la app móvil (CU36)		
PK `usuarioId`	S	
SK `dispositivoId`	S	
`tokenFcm`	S	Token de Firebase Cloud Messaging
`plataforma`	S	ANDROID | IOS
`activo`	BOOL	Se desactiva si FCM lo rechaza
`ultimoUso`	S	

EventosProcesados Idempotencia de RabbitMQ (CU36)		
PK `eventoId`	S	RabbitMQ puede entregar el mismo evento más de una vez
`procesadoEn`	S	
`ttl`	N	Expira a los 7 días

7. Asistente IA **DynamoDB · Python (FastAPI)**

Sesiones, mensajes y ayudas por foto. Las imágenes del tablero no se guardan en la tabla: va la clave del archivo en el storage. La IA local del móvil (CU41) corre en el dispositivo y no necesita tabla.

SesionesIA Pruebas de conducción asistidas (CU39)		
PK `usuarioId`	S	
SK `sk`	S	'<fecha ISO>#<sesionId>'
`sesionId`	S	ULID
`tipo`	S	PRUEBA_CONDUCCION | CHAT
`categoria`	S	Opcional
`escenario`	S	Opcional
`estado`	S	EN_CURSO | FINALIZADA
`puntaje`	N	Opcional
`retroalimentacion`	S	Opcional
`modelo`	S	Modelo usado
`duracionSeg`	N	Opcional

MensajesIA Conversación de una sesión (CU39)		
PK `sesionId`	S	
SK `seq`	N	Número de mensaje: ordena la conversación
`rol`	S	USER | ASSISTANT
`contenido`	S	
`mediaKey`	S	Opcional. Archivo en el storage
`ts`	S	
Índices y reglas: Se guarda aparte de SesionesIA para no pasar el límite de 400 KB por ítem.		

AyudasTablero Ayuda por foto del tablero (CU40, CU42)		
PK `usuarioId`	S	
SK `sk`	S	'<fecha ISO>#<ayudaId>'
`imagenKey`	S	Clave del archivo en el storage
`indicadores`	L	[{ codigo, nombre, confianza, severidad }]
`respuesta`	S	Explicación para el estudiante
`modelo`	S	
`latenciaMs`	N	
`util`	BOOL	Opcional. Opinión del estudiante

AgentesIA Agentes y flujos de n8n (CU43)		
PK `agenteId`	S	
`nombre`	S	
`descripcion`	S	
`workflowN8n`	S	Id del flujo en n8n
`webhookUrl`	S	
`parametros`	M	Opcional
`activo`	BOOL	
`actualizadoPor`	S	usuarioId del administrador
Índices y reglas: n8n guarda su propio estado interno; aquí solo se registra qué agentes expone DriveHub.		

8. Plataforma: estado de la saga DynamoDB · Orquestador (Spring Boot)

El orquestador de la saga reserva-pago registra cada paso para poder retomar o compensar después de un fallo. GraphQL (CU45) y MCP (CU46) no tienen base propia: leen de los servicios.

SagaEstado Estado de cada saga (CU44, CU14, CU31)		
PK `sagaId`	S	ULID
`tipo`	S	RESERVA_PAGO
`estado`	S	INICIADA | PAGO_PENDIENTE | COMPLETADA | COMPENSANDO | COMPENSADA | FALLIDA
`pasos`	L	[{ nombre, estado, ts, error }]
REF `reservaId`	S	Académico.reservas._id
REF `pagoId`	S	Pagos.pago.id
REF `estudianteId`	S	
`version`	N	Bloqueo optimista: la actualización exige la versión leída
`creadoEn`	S	
`actualizadoEn`	S	
`ttl`	N	Expira a los 90 días de terminar
Índices y reglas: GSI 'PorEstado': PK estado, SK actualizadoEn. Un job lo consulta para detectar sagas atascadas (por ejemplo PAGO_PENDIENTE desde hace demasiado tiempo) y compensarlas.		

9. Referencias entre servicios

Estos son los campos que apuntan a otro servicio. Se validan al escribir (llamada REST al servicio dueño) o se mantienen por eventos.

Campo	Dueño del dato	Se usa en
`usuarioId`	Identidad.usuario.id	Escuela (estudiante, instructor), Contenido (publicadoPor, autorId), Notificaciones y Asistente IA (usuarioId)
`estudianteId / instructorId`	Escuela.estudiante.usuario_id / instructor.usuario_id (mismo UUID que usuarioId)	Académico (reservas, clases, exámenes...), Pagos (inscripcion_plan, pago), SagaEstado
`vehiculoId`	Escuela.vehiculo.id	Académico (plantillas, franjas, reservas, clases, exámenes)
`categoria / categoria_codigo`	Escuela.categoria_licencia.codigo	Académico (preguntas, simulacros, solicitudes), Pagos (plan_pago), Contenido (manuales, tutoriales)
`reserva_id / reservaId`	Académico.reservas._id	Pagos (pago), SagaEstado
`pago_id / pagoId`	Pagos.pago.id	Académico (reservas), SagaEstado
`sagaId`	SagaEstado.sagaId	Académico (reservas), Pagos (pago, reversion_pago)

10. Casos de uso y entidades

Qué entidades toca cada caso de uso, para comprobar que el modelo los cubre todos.

CU	Caso de uso	Servicio	Entidades
CU01	Registrar usuario	Identidad	usuario, usuario_rol (y estudiante o instructor en Escuela)
CU02	Iniciar sesión	Identidad	usuario, refresh_token
CU03	Cerrar sesión	Identidad	refresh_token (revocado)
CU04	Recuperar contraseña	Identidad	token_recuperacion, usuario
CU05	Gestionar roles y permisos	Identidad	rol, permiso, usuario_rol, rol_permiso
CU06	Gestionar perfil	Identidad	usuario

CU	Caso de uso	Servicio	Entidades
CU07	Gestionar autos	Escuela	vehiculo, mantenimiento_vehiculo
CU08	Gestionar estudiantes	Escuela	estudiante, asignacion_alumno
CU09	Gestionar docentes	Escuela	instructor, instructor_categoria
CU10	Asignar auto a instructor	Escuela	asignacion_vehiculo
CU11	Consultar disponibilidad	Escuela y Académico	vehiculo, instructor + franjas
CU12	Gestionar horarios del instructor	Académico	plantillas_horario, franjas
CU13	Solicitar clase	Académico	solicitudes_clase
CU14	Reservar horario	Académico	franjas, reservas, SagaEstado, outbox_eventos
CU15	Cancelar o reprogramar reserva	Académico	reservas (historialEstados), franjas
CU16	Consultar horario de clases	Académico	franjas, reservas
CU17	Gestionar clases	Académico	clases, reservas
CU18	Gestionar prueba de clases	Académico	pruebas_clase, resultados
CU19	Gestionar banco de preguntas	Académico	banco_preguntas
CU20	Configurar simulacro escrito	Académico	simulacros_config
CU21	Rendir simulacro escrito	Académico	intentos_simulacro, resultados
CU22	Gestionar examen de conducción	Académico	examenes_conduccion, resultados
CU23	Consultar resultados y notas	Académico	resultados
CU24	Simulacro de conducción (por verse)	Pendiente	Sin diseño todavía (partidas_minijuego, si se aprueba)
CU25	Gestionar plan de pagos y cuotas	Pagos	plan_pago, inscripcion_plan, cuota
CU26	Realizar pago	Pagos	pago, transaccion_pasarela
CU27	Confirmar pago (pasarela)	Pagos	transaccion_pasarela, pago, cuota, outbox_evento
CU28	Consultar historial de pagos	Pagos	pago, cuota, inscripcion_plan
CU29	Consultar pagos de mis alumnos	Pagos y Escuela	pago + asignacion_alumno
CU30	Generar comprobante	Pagos	comprobante
CU31	Revertir pago	Pagos	reversion_pago, pago, SagaEstado
CU32	Gestionar manuales	Contenido	manuales
CU33	Gestionar tutoriales	Contenido	tutoriales
CU34	Consultar manuales	Contenido	manuales
CU35	Consultar tutoriales	Contenido	tutoriales
CU36	Enviar notificación	Notificaciones	Notificaciones, DispositivosPush, EventosProcesados, PreferenciasNotificacion
CU37	Configurar preferencias	Notificaciones	PreferenciasNotificacion
CU38	Historial de notificaciones	Notificaciones	Notificaciones (índice NoLeidas)
CU39	Prueba de conducción con IA	Asistente IA	SesionesIA, MensajesIA
CU40	Analizar foto del tablero	Asistente IA	AyudasTablero
CU41	Asistente IA local	Dispositivo	Sin tabla en el servidor (opcional: SesionesIA tipo CHAT)
CU42	Historial de ayudas IA	Asistente IA	AyudasTablero
CU43	Administrar agentes y flujos n8n	Asistente IA	AgentesIA
CU44	Orquestar saga de reserva y pago	Plataforma	SagaEstado, outbox_evento, outbox_eventos
CU45	Consultar datos vía GraphQL	Plataforma	Sin base propia

CU	Caso de uso	Servicio	Entidades
CU46	Herramientas de la plataforma por MCP	Plataforma	Sin base propia

11. Decisiones a confirmar

- **Persona única:** usar el mismo UUID en todos los servicios evita tablas de equivalencias, pero ata todos los identificadores a Identidad.
- **Instructor titular:** `asignacion_alumno` permite un solo instructor vigente por estudiante. Si un estudiante puede tomar clases con varios instructores a la vez, se quita ese índice único y 'mis alumnos' (CU29) se calcula desde las clases.
- **Dueño de la saga:** `SagaEstado` asume un orquestador propio. Si la saga vive dentro de Académico, la tabla pasa a ser de ese servicio.
- **Simulacro de conducción (CU24):** sigue sin diseño porque está marcado como 'por verse'.